Princess Sumaya University for Technology
جامعة الأميرة سمية للتكنولوجيا

Computer Science Department

First Exam

CS-11313: Algorithm Analysis and Design

Fall Semester 2022/2023
Thursday, 10-Nov-22@ 3:00 PM

Student Name:		**Student Number:**
Instructor:	**Section # (or time):**	**Signature:**

Instructions

- Time allowed: **60 Minutes**
- This booklet should be 6 pages long, including this page. If not, report this to the instructor.
- Answer all questions in the examination paper provided.
- Write your answers clearly: you may not receive full credit if your answer is unreadable.

Question	Max Mark	Your Mark	CLO
1	**33**		CLO 1
2	**34**		CLO 2
3	**33**		CLO 1
Total	**100**		

Question 1: Recurrences [33 Points]

a) Express the following code samples using a recurrence equation (13 points)

Function1(a[], n)	Function2(a[], n, k)

if n == 1:
  return
 	
min = findMin(a,n)
print(min)
 	
Function1(a, n/2)
Function1(a, n/2)
Function1(a, n/2)
	if n == 1:
	  return
	
	temp = a[0]
	if temp > 10
	  min = findMin(a, n)
	  Function2(a, n/2, min)
	else
	  max = findMax(a, n)
	  Function2(a, n/2, max)

Recurrence Equation for Function1 ➔ $T(n) = 3T\left(\frac{n}{2}\right) + n$	Recurrence Equation for Function2 ➔ $T(n) = T\left(\frac{n}{2}\right) + n$

b) Solve the following recurrence equations using the Recursion Tree method. Show all steps for full credit. (10 points for each)

$T(n) = T(n/3) + c \quad \text{if } n > 1, \quad 1 \text{ if } n \le 1$

$a = 1 \quad b = 3, \; H = \log_3 n$

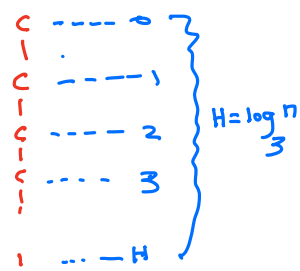
nodes per level $= 1^i = 1$

work per node $= c$

" " level $= 1 \cdot c$

Total work $= \sum_{i=0}^{\log_3 n} c = c \cdot (\log_3 n + 1)$

$= \Theta(\log n)$



$T(n) = 2T(n/16) + \sqrt{n} \quad \text{if } n > 1, \quad 1 \text{ if } n \le 1$

$a = 2, \; b = 16, \; H = \log_{16} n$

nodes per level $i = 2^i$

work per node $= \sqrt{\frac{n}{16^i}}$

work per level $i = 2^i \cdot \sqrt{\frac{n}{16^i}}$

Total work all levels

$= \sum_{i=0}^{\log_{16} n} 2^i \cdot \sqrt{\frac{n}{16^i}} = \sum_{i=0}^{\log_{16} n} 2^i \cdot \sqrt{\frac{n}{(4^2)^i}}$

$= \sum_{i=0}^{\log_{16} n} 2^i \cdot \sqrt{\frac{n}{4^i \cdot 4^i}} = \sum_{i=0}^{\log_{16} n} 2^i \cdot \frac{\sqrt{n}}{4^i} = \sum_{i=0}^{\log_{16} n} \cancel{2^i} \cdot \frac{\sqrt{n}}{\cancel{4^i}\, 2^i}$

$= \sqrt{n} \sum_{i=0}^{\log_{16} n} \left(\frac{1}{2}\right)^i = \sqrt{n} \cdot \left(\frac{\frac{1}{2}^{\log_{16} n + 1} - 1}{\frac{1}{2} - 1}\right) = \sqrt{n}\left(\frac{\left(\frac{1}{2}\right)^{\log_{16} n} \cdot \frac{1}{2} - 1}{-\frac{1}{2}}\right)$

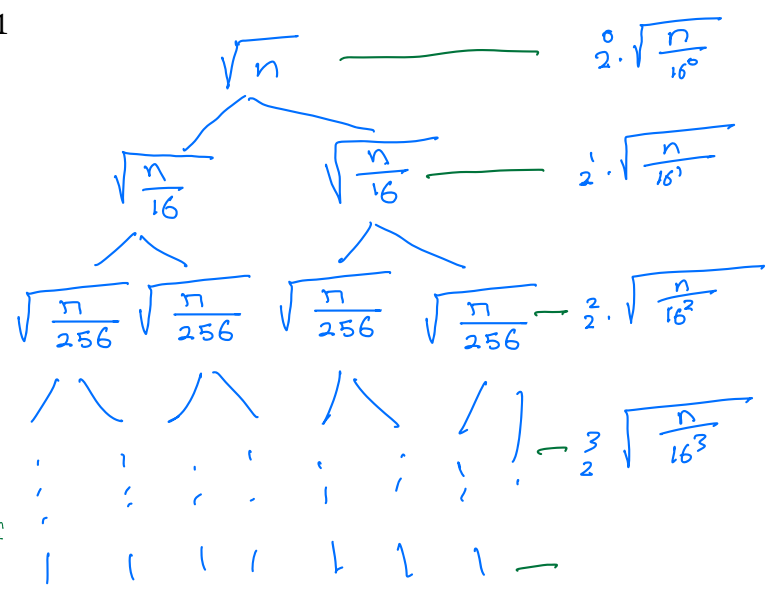
$= \sqrt{n}\left(2 - \frac{1}{2^{\log_{16} n}}\right)$ ⊙ simplify $\log_{16}$ $\rightarrow$ $\log_{16} n = \frac{\log_2 n}{\log_2 16} = \frac{\log_2 n}{\log_2 2^4} = \frac{1}{4} \cdot \log_2 n$

$\frac{1}{2^{\frac{1}{4} \cdot \log_2 n}} = 2^{-\log n^{0.25}} = n^{-0.25}$

$= \sqrt{n}\left(2 - \frac{4}{n^{0.25}}\right)$

$= 2\sqrt{n} - \frac{4\sqrt{n}}{n^{0.25}} = 2n^{0.5} - 4n^{0.25}$

$= \Theta(n^{0.5}) = \Theta(\sqrt{n})$



Question 2: Divide and Conquer [34 Points]

Consider the following code for QuickSort algorithm,

```
1  QUICKSORT(a[], first, last)
2  if first >= last:
3    return
4
5  p = PARTITION(a, first, last)
6
7  QUICK-SORT (a, first, p - 1)
9  QUICK-SORT (a, p + 1, last)
   `
```

```
PARTITION(a[], first, last)
pivot = a[first], i = first, j = last+1

repeat forever:
  do i = i+1
  while (i < last and a[i] < pivot)
  do j = j-1
  while (j > first and a[j] > pivot)

  if i >= j: break
  else: swap(a[i], a[j])

swap(a[first], a[j])

cout << " indices = " << first << " & "
    << j << " values = " << a[first]
    << " & " << a[j] << endl;
return j
```

a) (10 points) What is the asymptotic complexity of the worst case for the QuickSort algorithm expressed in **θ** notation? $\theta(n^2)$

When does it happen? Give an example on the worst case with an array of size 7?

When data is sorted (Ascending or Descending), an example array {1,2,3,4,5,6,7}

Write the recurrence equation of the QuickSort Algorithm in the worst case?

T(n)= T(n-1) + n

b) Print the output generated from below code: (8 points)

```
int a[] = {4, 8, 7, 1, 3, 5, 6};
QuickSort(a, 0, 6);
```

```
indices= 0 & 2 values= 1& 4
indices= 0 & 0 values= 1 & 1
indices= 3 & 5 values= 5& 7
indices= 3 & 3 values= 5 & 5
```

c) Rewrite the QuickSort function to check if any of the partitions is empty and to repartition only once in a way that guarantees having two non-empty partitions. These statements should be applied only if the current size of the array is 8 or larger. Assume that the data has no duplicate values. You are expected to add your code between lines 5 and 7 in the QuickSort function.

What is the impact of your new code on the runtime of the algorithm. (8 points)

```
if (last - first >= 8)
  if (p - 1 < first)
      if (data[p + 1] < data[p + 2])
          Swap(data[p], data[p+1]);
      else
          Swap(data[p], data[p + 2]);
  else if (p + 1 > last)
      if (data[p - 1] < data[p - 2])
          Swap(data[p], data[p - 1]);
      else
          Swap(data[p], data[p - 2]);
  p = Partition(data, first, last);
```

d) Write a new divide and conquer algorithm called **RangeQuickSort** that sorts the array values that fall within the range [A .. B] inclusively. Assume that the values A and B exist in the array. The algorithm sorts *only* the values that are within the given range. Assume the above Partition function is written and available to use/call. Assume no duplicate values exist. (8 points)

Example:

```
int a[] = {14, 98, 16, 12,  15,5, 88, 2, 20, 33}
RangeQuickSort(a, 0, 9, 14, 20)
```

`Should return array with the range 14 to 20 sorted`

Value	5	12	2	**14**	**15**	**16**	**20**	88	33	98
index	0	1	2	3	4	5	6	7	8	9

Notice that the values to the left of the range and the values to the right of the range could be of any order and do not have to be sorted. Your algorithm must be more efficient than sorting the whole array.

```
void RangeQuickSort(int data[], int first, int last, int A, int B) {
 if (last <= first)
     return;
 int p = Partition(data, first, last);
 if (data[p] <= A)
      RangeQuickSort(data, p + 1, last, A, B);
 else if (data[p] >= B)
      RangeQuickSort(data, first, p - 1, A, B);
 else  {
      RangeQuickSort(data, first, p - 1, A, B);
      RangeQuickSort(data, p + 1, last, A, B);
 }
}
```

Question 3: Complexity Analysis [33 Points]

1. Assume $f(n) = O(n^2)$ and $f(n) = O(n^3)$, which of the following functions could $f(n)$ be? Mark all that applies. (6 points)

(a) n (b) n^2 (c) n^3 (d) $n^{3.5}$

2. Assume that $f(n) = n^2 + O(n)$, which of the following is true about f(n)? Mark all that applies. (6 points)

(a) $f(n) = O(n)$ (b) $f(n) = \Omega(n)$ (c) $f(n) = O(n^2)$

(d) $f(n) = \Omega(n^2)$ (e) $f(n) = \Omega(n^3)$ (f) $f(n) = O(n^3)$

3. Provide an example for the function $f(n)$ in each of the following cases. (9 points)

(a) If $f(n) = \omega(n^k)$ and $f(n) = o(3^n)$, where k > 0. **$f(n) = o(2^n)$**

(b) If $f(n) = \omega(1)$ and $f(n) = o(\sqrt{n})$ $f(n)=\log(n)$

(c) If $f(n) = \omega(\log(n))$ and $f(n) = o(n^k)$, where $0 < k \leq 1$. $f(n)=\log(n)\log(n)$

4. Give a counter example that shows that the following statement is not always true: (4 points

if $f(n) = \Omega(g(n))$ and $g(n) = \Omega(h(n))$ then $f(n) = \omega(h(n))$

if $f(n)=g(n)=h(n)=n^3$, then $f(n) = \Omega(g(n))$ and $g(n) = \Omega(h(n))$ is true
but $n^3 \neq \omega(n^3)$

5. Prove that $kn + k = O\left(\frac{n}{k}\right)$ for any $k > 0$. (8 points)

see next page

Big-O Definition

$f(n) = O(g(n))$ iff there are two constants

c and n_0 such that

$0 \leq f(n) \leq c \cdot g(n)$ for all $n \geq n_0$

$f(n) = kn + kn$, $g(n) = \frac{n}{k}$

$kn + k \leq c \cdot \frac{n}{k}$

$\Rightarrow \quad kn + kn \leq c \cdot \frac{n}{k}$

$2kn \leq c \cdot \frac{n}{k}$

pick $c = 2k^2 \Rightarrow$

$2kn \leq 2 \cdot k^2 \cdot \frac{n}{k}$

$2kn \leq 2kn$ for $n_0 = 1$

$\Rightarrow \boxed{c = 2k^2, \; n_0 = 1}$